

INFORME FINAL - PROYECTO CALIDAD DE SOFTWARE

Planteamiento del problema

Problema general

La gestion manual de la asistencia y los retardos de los empleados provoca errores de registro, dificulta el seguimiento de horarios y reduce la confiabilidad de los reportes administrativos.

Problemas especificos

- El registro manual de entradas y salidas puede generar omisiones y datos inconsistentes.
- No existe trazabilidad suficiente para identificar tardanzas, ausencias y observaciones.
- La elaboracion de reportes consume tiempo y depende de procesos repetitivos.
- La informacion puede quedar dispersa en hojas de calculo o formularios sin control centralizado.

Contexto

La empresa o institucion necesita un sistema centralizado que almacene los datos de asistencia en una base de datos, permita consultar historiales por empleado y genere reportes utiles para control administrativo y toma de decisiones.

Objetivos

Objetivo general

Desarrollar un sistema web para el control de asistencia y retardos de empleados con base de datos, pruebas automatizadas y despliegue en línea, aplicando practicas de SDD y Scrum.

Objetivos especificos

- Analizar las necesidades funcionales y no funcionales del proceso de control de asistencia.
- Diseñar la estructura del sistema y el modelo de datos para registrar empleados, horarios y asistencias.
- Implementar una aplicacion web que permita registrar entradas, salidas, tardanzas y reportes.
- Incorporar pruebas unitarias e integracion para validar reglas de negocio y flujo entre modulos.
- Desplegar la aplicacion y documentar la evidencia tecnica del proyecto.

Resumen ejecutivo

El presente proyecto propone el desarrollo de un sistema web para el control de asistencia y retardos de empleados, respaldado por una base de datos PostgreSQL en Supabase y desplegado en Vercel. La solución responde a la necesidad de reemplazar registros manuales o dispersos por un sistema centralizado, trazable y fácil de consultar.

El trabajo se desarrolló bajo un enfoque de SDD y Scrum. Primero se definieron el problema, los objetivos, la justificación, las limitaciones y el alcance. Posteriormente se diseñó el modelo de datos, se organizaron los entregables en sprints y se implementó un MVP funcional con módulos de empleados, turnos, asistencia, roles y reportes.

Como parte de la calidad del software, se incluyeron pruebas unitarias para validar reglas de tardanza, cálculo de tiempo trabajado y resumen de asistencias, así como pruebas de integración orientadas a la persistencia y consulta de datos mediante rutas API. También se preparó un esquema SQL, datos semilla y una estrategia de despliegue reproducible.

En conjunto, el proyecto demuestra el ciclo de vida completo del software y genera evidencia útil para la presentación académica, el informe final en Word, el repositorio en GitHub y la exposición oral.

Justificacion

El proyecto se justifica porque automatiza un proceso frecuente y de alto impacto en cualquier organizacion: el control de asistencia. Al reemplazar registros manuales por un sistema web con base de datos, se reducen errores, se mejora la trazabilidad y se facilita la consulta de informacion historica.

Desde la perspectiva academica, el proyecto permite demostrar el ciclo de vida completo del software, desde el analisis y diseno hasta la programacion, pruebas y despliegue. Ademias, facilita evidencias claras de SDD y Scrum, ya que cada parte del sistema puede documentarse con requisitos, historias de usuario, entregables por sprint y pruebas verificables.

Limitaciones y alcance

Alcance

El sistema incluiría el registro de empleados, horarios, entradas, salidas, tardanzas y reportes básicos. También contemplaría roles de acceso y una base de datos centralizada para conservar la información.

Limitaciones

- No incluye biometría ni hardware especializado.
- No cubre nómina ni cálculo salarial.
- No integra geolocalización ni validación avanzada por dispositivo.
- No contempla analítica compleja o inteligencia artificial en la primera versión.
- El alcance inicial se enfoca en un producto mínimo viable que sea demostrable y defendible en el tiempo disponible.

Marco teorico

Ciclo de vida del software

El ciclo de vida del software incluye analisis, diseno, programacion, pruebas, despliegue y mantenimiento. En este proyecto se documenta cada fase para mostrar como la solucion evoluciona desde el problema hasta una aplicacion usable.

SDD

SDD se entiende aqui como un enfoque de desarrollo guiado por especificaciones y diseno previo. Primero se definen requisitos, reglas de negocio y criterios de aceptacion; luego se diseña la solucion y finalmente se implementa y verifica contra lo especificado.

Scrum

Scrum organiza el trabajo en incrementos cortos llamados sprints. Para este proyecto, Scrum sirve para dividir el trabajo en entregables pequenos: analisis, diseno, base de datos, implementacion, pruebas y despliegue.

Pruebas de software

Las pruebas unitarias verifican funciones o reglas individuales. Las pruebas de integracion comprueban que los modulos trabajen juntos correctamente, por ejemplo, al registrar una asistencia y guardar el dato en la base de datos.

Despliegue

El despliegue permite publicar la aplicacion en un entorno accesible para la demo final. Para este proyecto se propone Vercel para la app web y Supabase para la base de datos PostgreSQL.

Materiales y metodos

Materiales

- Editor de codigo: Visual Studio Code
- Control de versiones: Git y GitHub
- Frontend: Next.js con TypeScript
- Base de datos: PostgreSQL en Supabase
- Despliegue: Vercel
- Pruebas: Vitest y Testing Library

Metodos

1. Levantamiento de requisitos a partir del problema definido.
2. Elaboracion de historias de usuario y criterios de aceptacion.
3. Diseno del modelo de datos y reglas de negocio.
4. Implementacion incremental del sistema.
5. Elaboracion y ejecucion de pruebas.
6. Despliegue y documentacion de resultados.

Analisis y diseno bajo SDD

Vision de producto

El sistema permite centralizar el control de asistencia y retardos de empleados en una aplicacion web con base de datos. El usuario principal puede registrar personal, definir turnos, capturar asistencias y revisar reportes historicos sin depender de hojas de calculo manuales.

Requisitos funcionales

- RF01: Registrar empleados con codigo, nombre, correo y estado.
- RF02: Registrar turnos con hora de inicio, hora de fin y tolerancia.
- RF03: Registrar entradas y salidas de asistencia por empleado y fecha.
- RF04: Calcular minutos de tardanza segun el turno asignado.
- RF05: Consultar historial de asistencia por empleado.
- RF06: Generar reportes basicos por rango de fechas.
- RF07: Gestionar acceso por roles para administrador y supervisor.

Requisitos no funcionales

- RNF01: La aplicacion debe ser accesible desde navegador web.
- RNF02: La informacion debe persistirse en PostgreSQL.
- RNF03: La logica critica debe contar con pruebas unitarias.
- RNF04: Los modulos principales deben validar el flujo de datos con pruebas de integracion.
- RNF05: La aplicacion debe poder desplegarse en un entorno publico.

Historias de usuario

- HU01: Como administrador, quiero registrar empleados para tener control del personal.
- HU02: Como administrador, quiero definir turnos para comparar la hora real con la hora programada.
- HU03: Como encargado, quiero registrar entradas y salidas para llevar asistencia diaria.
- HU04: Como supervisor, quiero ver tardanzas acumuladas para detectar incumplimientos.
- HU05: Como supervisor, quiero consultar reportes por fecha para revisar comportamiento historico.

Criterios de aceptacion

- CA01: El sistema guarda empleados y turnos en la base de datos.
- CA02: El sistema no permite duplicar registros de asistencia para el mismo empleado y fecha.
- CA03: El sistema calcula tardanza cuando la hora de ingreso supera la hora del turno.
- CA04: El sistema muestra historiales y reportes con datos consistentes.
- CA05: Las reglas criticas deben pasar pruebas automatizadas.

Casos de uso resumidos

Caso de uso: Registrar empleado

Actor principal: administrador. Resultado esperado: el empleado queda disponible para los modulos de asistencia y reportes.

Caso de uso: Registrar asistencia

Actor principal: encargado. Resultado esperado: la asistencia se guarda y se calcula la tardanza asociada al turno.

Caso de uso: Consultar reportes

Actor principal: supervisor. Resultado esperado: el sistema muestra un resumen de asistencia por rango de fechas.

Trazabilidad inicial

- RF01 -> tabla employees
- RF02 -> tabla shifts
- RF03 -> tabla attendance_records
- RF04 -> funcion calculateTardinessMinutes
- RF05 -> consulta por employee_id sobre attendance_records
- RF06 -> componente o endpoint de reportes
- RF07 -> tablas roles y profiles

Suposiciones de diseno

- La hora de referencia se maneja en formato de 24 horas.
- La tardanza se calcula en minutos enteros.
- Cada empleado puede tener un unico registro por fecha.
- El sistema inicial no incluye biometria ni hardware externo.

Gestion del proyecto con Scrum

Product backlog inicial

1. Registrar empleados.
2. Registrar horarios.
3. Registrar entradas y salidas.
4. Calcular tardanzas.
5. Generar reportes basicos.
6. Desplegar la aplicacion.

Sprints propuestos

- Sprint 1: analisis del problema, alcance y requisitos.
- Sprint 2: diseno de base de datos y estructura tecnica.
- Sprint 3: implementacion del modulo de registro.
- Sprint 4: pruebas, ajustes y despliegue.

Entregables por sprint

- Documentacion en Markdown.
- Modelo de datos.
- Codigo fuente funcional.
- Evidencia de pruebas.
- Evidencia de despliegue.

Diseno de base de datos

Esquema de Base de Datos

El sistema está respaldado por PostgreSQL (Supabase) con las siguientes 4 tablas principales:

1. employees (Empleados)

Almacena la información del personal.

- id (uuid, PK)
- employee_code (text, único)
- full_name (text)
- email (text)
- active (bool)
- created_at (timestampz)

2. attendance_records (Registros de Asistencia)

Almacena el registro diario de cada empleado.

- id (uuid, PK)
- employee_id (uuid, FK -> employees.id)
- work_date (text)
- clock_in (text)
- clock_out (text, opcional)
- tardiness_minutes (int4)
- notes (text, opcional)
- shift_id (text, FK -> shifts.id)
- created_at (timestampz)

3. shifts (Turnos)

Define los horarios laborales aplicables.

- id (text, PK)
- name (text)
- start_time (text)
- end_time (text)
- tolerance_minutes (int4)
- created_at (timestampz)

4. roles (Roles)

Define los roles del sistema para el acceso (ej. Admin, Supervisor).

- id (text, PK)
- name (text)
- created_at (timestampz)

Relaciones Principales

- Un Empleado (employees) tiene muchos Registros de Asistencia (attendance_records). (Relación 1:N a través de employee_id).
- Un Turno (shifts) puede estar asociado a múltiples Registros de Asistencia (attendance_records). (Relación 1:N a través de shift_id).

Reglas de negocio

- Cada empleado debe tener un identificador unico.
- Un registro de asistencia no debe duplicarse para el mismo empleado y fecha.
- La tardanza se calcula comparando la hora de ingreso con el horario del turno.
- Los reportes deben basarse en datos guardados en la base de datos y no en valores manuales.

Pruebas unitarias e integracion

Pruebas unitarias

- Calculo de tardanza.
- Clasificacion de asistencia puntual o tardia.
- Resumen de minutos trabajados.
- Validacion de bordes como reloj de salida anterior al de entrada.
- Resumen vacio cuando no existen registros.

Pruebas de integracion

- Registro de asistencia y persistencia en la base de datos.
- Consulta de historial por empleado.
- Generacion de reportes con datos reales.
- Registro de empleados con la ruta `/api/employees` .
- Registro de turnos con la ruta `/api/shifts` .
- Verificacion de roles con la ruta `/api/roles` .

Evidencia esperada

- Casos de prueba escritos.
- Ejecucion automatizada en la terminal.
- Resultados correctos sin fallos criticos.

Criterios de exito

- Las funciones puras deben devolver el resultado esperado en situaciones normales y bordes.
- Las rutas API deben aceptar datos validos y rechazar campos incompletos.
- El reporte debe conservar la consistencia entre totales, tardanzas y puntualidad.

Evidencia de Ejecución (Captura)

A continuación, se presenta la captura de la ejecución real de la suite de pruebas unitarias y de integración utilizando **Vitest**, demostrando que el 100% de los casos (10/10) se ejecutaron con éxito.

```
> proyecto-asistencia-empleados@0.1.0 test
> vitest run

 RUN  v3.2.7 C:/Users/luisg/OneDrive/Documentos/Proyecto final calidad de software/proyecto-asistencia-
empleados

 ✓ tests/attendance.test.ts (6 tests) 11ms
 ✓ tests/report.test.ts (2 tests) 12ms
 ✓ tests/api-integration.test.ts (2 tests) 76ms

Test Files  3 passed (3)
   Tests    10 passed (10)
   Start at 21:30:42
   Duration 3.49s (transform 317ms, setup 0ms, collect 717ms, tests 99ms, environment 6.18s, prepare
939ms)
```

Como se observa, se evaluaron correctamente los componentes de asistencia, reportes y la integración nativa de la API sin fallos detectados.

Despliegue de la aplicacion

Estrategia propuesta

- Frontend y despliegue: Vercel.
- Base de datos: Supabase PostgreSQL.
- Variables de entorno: credenciales de Supabase.
- Datos semilla: `supabase/seed.sql` para cargar un escenario de demostracion.

Flujo de publicacion

1. Crear el proyecto en Supabase y ejecutar `supabase/schema.sql` .
2. Cargar `supabase/seed.sql` para disponer de datos de prueba.
3. Configurar las variables de entorno en Vercel.
4. Publicar la aplicacion desde el repositorio de GitHub.
5. Verificar las rutas `/api/employees` , `/api/shifts` , `/api/attendance` y `/api/roles` .

Evidencia requerida

- URL publica de la aplicacion.
- Capturas del sistema funcionando.
- Capturas o registros del panel de base de datos.
- Descripcion breve de configuracion y publicacion.

Resultados

Resultados esperados

- Sistema capaz de registrar empleados, turnos, asistencias y roles desde una interfaz web.
- Base de datos estructurada y consultable mediante rutas API.
- Pruebas automatizadas ejecutadas correctamente para reglas de tardanza, resumen y bordes.
- Aplicacion preparada para despliegue con Supabase y Vercel.

Indicadores de exito

- Reduccion de errores de registro al centralizar la informacion en una base de datos.
- Mejora de trazabilidad al relacionar empleados, turnos y asistencias.
- Generacion de reportes rapidos y consistentes desde datos persistidos.
- Evidencia clara de SDD, Scrum, pruebas y despliegue en la documentacion del proyecto.

Resultados obtenidos en el proyecto

- Se definio una estructura documental completa en Markdown para el informe final.
- Se implemento un MVP funcional con formularios para empleados, asistencia, turnos y roles.
- Se construyeron funciones puras para calcular tardanza, tiempo trabajado y resumen de asistencia.
- Se agregaron rutas API para consultar y registrar informacion con base de datos.
- Se preparo el esquema SQL y un archivo de semillas para una demostracion reproducible.

Evidencia tecnica

- Validacion de sintaxis y tipado sin errores en el subproyecto.
- Pruebas unitarias para casos normales y bordes.
- Componentes de interfaz conectados a rutas API.
- Documentacion de despliegue y flujo de publicacion.

Capturas de la Interfaz (UI)

A continuación, se presentan las capturas del resultado visual del rediseño del sistema:

1. Panel de Control (Dashboard)

El dashboard principal presenta las métricas del sistema y un resumen visual del estado de las tardanzas.

 Dashboard de Control

2. Gestión de Empleados y Registros

Las listas utilizan un componente de tablas avanzado con filtros, contadores dinámicos y "badges" de colores para distinguir estados (Ej. Puntual vs Tardanza).

 Lista de Empleados

3. Pantalla de Inicio de Sesión

El sistema cuenta con una pantalla de Login segura donde el empleado ingresa su código para acceder a su portal, separando así los roles administrativos de los usuarios finales.

 Inicio de Sesión

4. Portal del Empleado (Kiosko)

Un terminal de autoservicio que muestra la hora en tiempo real y permite al trabajador registrar su entrada y salida mediante botones interactivos, los cuales confirman la hora exacta de la marcación.

 Kiosko de Empleado

Conclusiones y recomendaciones

Conclusiones

El proyecto permite demostrar el desarrollo completo de una solución de software con base de datos, documentación formal, pruebas y despliegue. También deja evidencia clara de trabajo bajo SDD y Scrum, porque los requisitos, el diseño, la implementación y la verificación quedaron enlazados desde el inicio.

La solución propuesta es adecuada para un proyecto académico porque su alcance es suficiente para mostrar análisis, diseño, programación, pruebas y despliegue sin depender de hardware especializado ni de integraciones complejas. Esto facilita la defensa oral y la demostración técnica.

La estructura documental en Markdown permite construir el informe final en Word y la presentación en PPT sin rehacer el contenido. Además, el repositorio queda preparado para mostrar trazabilidad entre requisitos, base de datos, pruebas y resultados.

Recomendaciones

- Completar el guardado real en Supabase si el entorno de prueba ya está configurado con credenciales.
- Mantener trazabilidad entre requisitos, historias de usuario, tablas, rutas API y pruebas.
- Guardar capturas de la interfaz, del esquema de base de datos y de la ejecución de pruebas como soporte de defensa.
- Exportar la documentación final a Word y reutilizar el guion de PPT para la exposición.

Matriz de trazabilidad

Requisito	Historia de usuario	Modulo	Tabla / ruta	Prueba
Registrar empleados	HU01	Administracion de empleados	employees / /api/employees	Creacion de empleado
Definir turnos	HU02	Gestion de turnos	shifts / /api/shifts	Creacion de turno
Registrar asistencia	HU03	Registro de asistencia	attendance_records / /api/attendance	Tardanza y guardado
Calcular tardanzas	HU03, HU04	Registro de asistencia	calculateTardinessMinutes	Caso puntual y tardio
Consultar reportes	HU05	Reportes	/api/reports	Resumen de asistencia
Gestionar roles	HU07	Autenticacion y roles	roles / /api/roles	Registro de rol

Observacion

La matriz muestra como cada requisito funcional del proyecto se conecta con una historia de usuario, un modulo, una estructura de datos y una prueba automatizada o de integracion. Esto facilita demostrar SDD ante el docente.

Control de versiones

Objetivo

Organizar el desarrollo del proyecto mediante Git y GitHub para registrar avances, controlar cambios y conservar evidencia del trabajo incremental.

Estrategia de trabajo

- Repositorio remoto: GitHub.
- Rama principal: `main`.
- Trabajo de implementacion: cambios pequeños y frecuentes.
- Evidencia de avance: commits descriptivos y consistentes.

Flujo sugerido

1. Crear o actualizar una rama de trabajo cuando exista una tarea amplia.
2. Implementar un cambio acotado por vez.
3. Validar el cambio con pruebas o revision local.
4. Registrar el commit con un mensaje claro.
5. Subir el avance al repositorio remoto.

Convenciones de commit

- `feat` : nueva funcionalidad.
- `fix` : correccion de error.
- `docs` : cambios en documentacion.
- `test` : cambios en pruebas.
- `refactor` : mejora interna sin cambio funcional.

Evidencias para el docente

- Historial de commits en GitHub.
- Trazabilidad entre commits, sprints y entregables.
- Uso de ramas y mensajes descriptivos.
- Relacion entre codigo, pruebas y documentacion.

Beneficio academico

El control de versiones permite demostrar que el proyecto no fue desarrollado en un solo bloque, sino de forma incremental y organizada, lo cual refuerza la aplicacion de Scrum y el seguimiento del ciclo de vida del software.

Referencias

- The Scrum Guide. Schwaber, K. y Sutherland, J. 2020.
- Sommerville, I. Software Engineering. 10th edition.
- Documentacion oficial de Next.js: <https://nextjs.org/docs>
- Documentacion oficial de Supabase: <https://supabase.com/docs>
- Documentacion oficial de Vitest: <https://vitest.dev>
- Documentacion oficial de Testing Library: <https://testing-library.com/docs>
- Documentacion oficial de GitHub: <https://docs.github.com>

Checklist de entrega final

Documento Word

- Portada.
- Resumen ejecutivo.
- Planteamiento del problema.
- Objetivos.
- Justificacion.
- Limitaciones y alcance.
- Marco teorico.
- Materiales y metodos.
- Analisis y diseno bajo SDD.
- Gestion con Scrum.
- Diseno de base de datos.
- Implementacion.
- Pruebas.
- Despliegue.
- Resultados.
- Conclusiones.
- Referencias.
- Anexos.

Presentacion PPT

- Problema.
- Objetivos.
- Justificacion.
- Alcance.
- SDD y Scrum.
- Base de datos.
- Implementacion y pruebas.
- Despliegue.
- Resultados y conclusiones.

GitHub

- Codigo fuente.
- Esquema SQL.
- Datos semilla.
- Pruebas.
- Documentacion en Markdown.
- Capturas o evidencias de despliegue.

Verificacion final

- El proyecto debe compilar sin errores.
- Las pruebas deben pasar.
- La aplicacion debe estar desplegada o lista para desplegar.
- Las evidencias deben coincidir con lo documentado.

Defensa oral

Apertura

Buenos días. Presento el proyecto Sistema de control de asistencia y retardos para empleados, una solución web diseñada para reemplazar procesos manuales y mejorar la trazabilidad del registro de personal.

Problema

Actualmente, el control manual de asistencia genera errores, retrabajo y dificultad para consultar historiales de forma confiable. Por eso se propuso una aplicación centralizada con base de datos.

Solución propuesta

La solución incluye módulos de empleados, turnos, asistencia, roles y reportes. La información se organiza en Supabase PostgreSQL y la aplicación se despliega en Vercel para su demostración.

Metodología

El trabajo se desarrolló bajo SDD y Scrum. Primero se definieron requisitos, objetivos, justificación, limitaciones y trazabilidad. Luego se diseñó la base de datos, se implementó el MVP y se agregaron pruebas unitarias e integración.

Evidencia técnica

Se crearon rutas API para empleados, turnos, asistencia, roles y reportes. También se prepararon datos semilla, control de versiones en GitHub y una matriz de trazabilidad para relacionar requisitos, módulos y pruebas.

Resultados

El proyecto ya cuenta con una base funcional para registrar y consultar información, validar reglas de negocio y presentar evidencia académica clara en Word, PPT y GitHub.

Cierre

Como conclusión, el proyecto demuestra el ciclo de vida completo del software y deja evidencia de desarrollo incremental, control de versiones, pruebas y despliegue. Muchas gracias.

Indice general

1. Portada
2. Resumen ejecutivo
3. Planteamiento del problema
4. Objetivos
5. Justificacion
6. Limitaciones y alcance
7. Marco teorico
8. Materiales y metodos
9. Analisis y diseno bajo SDD
10. Gestion del proyecto con Scrum
11. Control de versiones
12. Diseno de base de datos
13. Implementacion del sistema
14. Pruebas unitarias e integracion
15. Despliegue de la aplicacion
16. Resultados
17. Conclusiones y recomendaciones
18. Referencias
19. Matriz de trazabilidad
20. Checklist de entrega final
21. Defensa oral
22. Anexos

Software Design Document (SDD) & Specifications

Proyecto: Sistema de Asistencia de Empleados (AsistControl) **Versión:** 1.0.0

1. Introducción

El presente documento (SDD) centraliza las especificaciones funcionales, técnicas y de arquitectura del proyecto AsistControl. Actúa como la fuente única de verdad (Single Source of Truth) para el desarrollo guiado por especificaciones (Specification-Driven Development).

2. Arquitectura del Sistema

El sistema sigue un modelo cliente-servidor desacoplado (Single Page Application):

- Frontend (Capa de Presentación):** Next.js (React) + Tailwind CSS.
- Backend (Capa de Negocio y Datos):** Next.js API Routes (Node.js).
- Validación de Especificaciones (Spec Kit):** Zod (Schema Validation).

3. Especificaciones de Funcionalidades (Specs)

Spec 01: Módulo de Kiosko (Marcación)

Actor: Empleado **Instrucción / Comportamiento Esperado:**

- El empleado ingresa al kiosko y visualiza la hora en tiempo real.
- El empleado presiona "Marcar Entrada" o "Marcar Salida".
- Condición de fallo (Negative Path):** Si el empleado ya registró una entrada en la fecha actual, el sistema debe bloquear una segunda entrada y mostrar una alerta roja.
- Condición de éxito (Happy Path):** Si la marcación es válida, el sistema muestra una alerta verde y persiste el registro en el archivo/base de datos.

Spec 02: Módulo de Autenticación

Actor: Administrador / Empleado **Instrucción / Comportamiento Esperado:**

- El usuario selecciona su cuenta o ingresa sus credenciales.
- Si el rol es `admin`, el sistema lo redirige a la ruta `/admin` tras validar el inicio de sesión.
- Si el rol es `empleado`, el sistema lo redirige a `/empleado` (Kiosko).

Spec 03: Panel de Reportes (Dashboard)

Actor: Administrador (Dueño) **Instrucción / Comportamiento Esperado:**

- El administrador visualiza tarjetas con métricas resumidas (Total empleados, Asistencias de hoy, Tardanzas).
- El administrador tiene acceso a una tabla con el registro detallado de las horas de llegada.
- Se calcula automáticamente si un empleado llegó "A tiempo" o "Tarde" basado en la hora límite (09:00 AM).

4. Contrato de la API (Open Spec)

Las rutas del servidor respetan el siguiente contrato de datos:

- Ruta:** `POST /api/attendance/mark`
- Cuerpo de la petición (Input):** `{ "employeeCode": "string", "type": "entrada" | "salida" }`
- Respuesta Exitosa:** `200 OK` con `{ "message": "..." }`
- Respuesta de Error:** `400 Bad Request` con `{ "error": "..." }`

5. Pruebas y Validación (QA)

- **Pruebas Unitarias (Jest):** Verifican matemáticamente las especificaciones de cálculo de tardanzas.
- **Pruebas E2E (Playwright):** Robot automatizado que lee este documento de especificaciones y verifica paso a paso que la interfaz gráfica cumpla con los comportamientos descritos en el *Spec 01* y *Spec 02*.

Anexos

- Capturas de la aplicacion principal mostrando empleados, turnos, asistencia, roles y reportes.
- Capturas de las rutas API o respuestas JSON para validar el flujo de datos.
- Evidencia de pruebas unitarias con resultados de la terminal.
- Capturas o enlaces del despliegue en Vercel.
- Capturas del esquema y tablas en Supabase.
- Historial de commits en GitHub como evidencia de trabajo incremental.
- Copia del esquema SQL y del archivo de semillas usados para la demostracion.